

MAD1 Report

1. Student Details

Name: Ranindram Dutta

Roll Number: 24F2003613

Email: 24f2003613@ds.study.iitm.ac.in

About Me: I am Ranindram Dutta, an Electrical Engineering student at Jadavpur University and a Data Science student at IIT Madras. I work at the intersection of technology, design, and data, focusing on web development, AI-driven applications, and smart systems. I enjoy building impactful solutions in areas such as health tech, sustainability, and intelligent automation, while combining strong UI/UX design with practical engineeri

2. Project Details

Project Title: Recruita

Problem Statement:

Currently, most institutes are using tools such as Excel or emails to manage campus recruitments. This becomes a problem while handling company approvals, student applications, duplicate registrations, etc.

Recruita will offer a central platform where Admin, Companies, and Students can interact with each other to facilitate the placement drives.

Approach:

Developed a role-based application portal with Flask web development, SQLite for the database, and Jinja2 templating. Designed a minimal normalized schema with strict backend checks for access control, approval, and preventing duplicate application attempts. Developed a module-based route/model implementation to ensure the application remains easy to explain, test, and extend.

3. AI/LLM Declaration

I used **ChatGPT (GPT-5)** to assist in for the flow of the project, routing APIs, and debugging few routings. I did not take design helps rather took suggestion on improving my db design. The extent of AI/LLM usage is around **17%**, limited to **code suggestions, improvements, debugging and documentation formatting**.

But ost debugging after suggestions and designing was mainly done by me took abit of help in routing in flask.

4. Technologies and Frameworks Used

Technology / Library	Purpose
Flask	Core backend web framework
Jinja2	Template engine for rendering dynamic HTML pages
Bootstrap 5	Frontend styling and responsive design
SQLite	Lightweight local database for storing user data

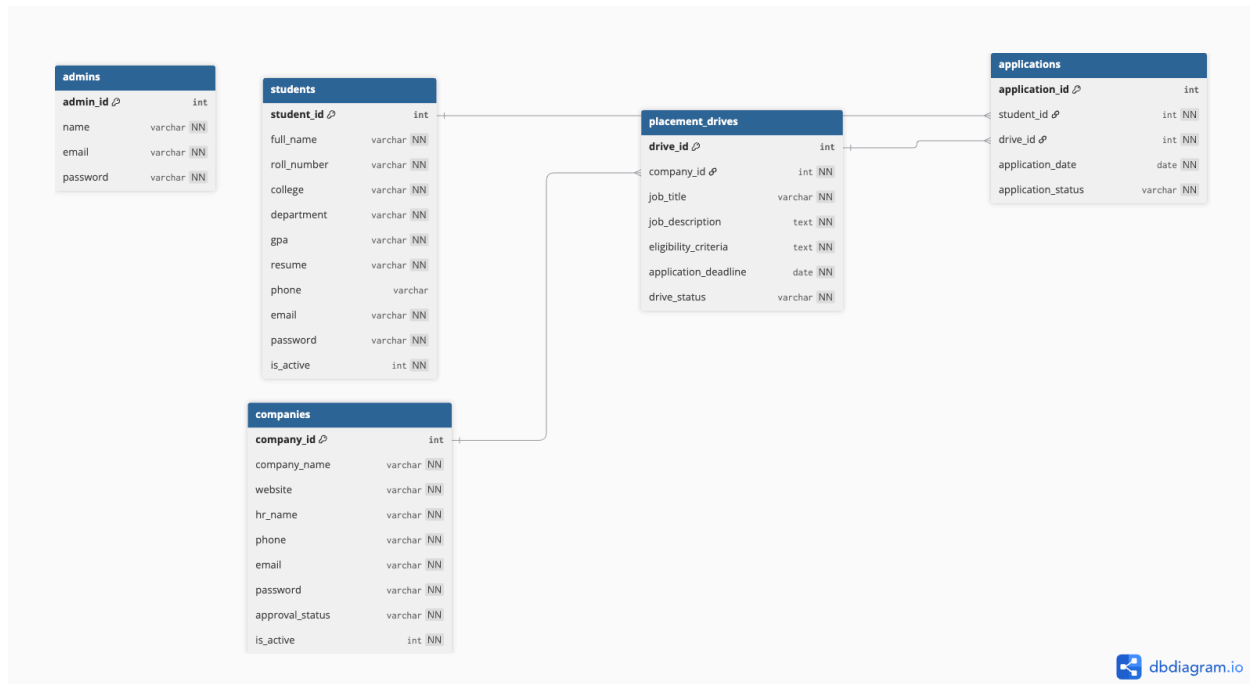
5. Database Schema / ER Diagram

Tables:

- **admins** — placement cell admin (admin_id, name, email, password) - present before
- **students** — student profiles and login data (student_id, full_name, roll_number, college, department, gpa, resume, phone, email, password, is_active)
- **companies** — recruiter profiles and approval state (company_id, company_name, website, hr_name, phone, email, password, approval_status, is_active)
- **placement_drives** — job/internship drives posted by companies (drive_id, company_id, job_title, job_description, eligibility_criteria, application_deadline, drive_status)
- **applications** — student applications to drives (application_id, student_id, drive_id, application_date, application_status)

Relationships:

- **One-to-Many** → companies → placement_drives
- **One-to-Many** → students → applications
- **One-to-Many** → placement_drives → applications



6. Architecture and Features

Architecture Overview:

app.py: Initializes Flask app, config, table creation call, and registers all route modules.

database.py: Manages SQLite connection and creates all core tables programmatically.

routes/

- **auth_routes.py:** login, logout, registration, role-based redirects.
- **admin_routes.py:** approval workflows, search, activation/blacklisting controls.
- **company_routes.py:** profile management, drive lifecycle, applicant review.
- **student_routes.py:** profile management, drive browsing, apply/history.

models/ : Contain role-wise SQL queries and DB actions

- **auth_models.py**
- **admin_models.py**
- **company_models.py**
- **student_models.py**

templates/: Jinja2 + Bootstrap pages split by role (auth, admin, company, student) with form-based actions.

- **admin/**
- **auth/**
- **company/**
- **student/**

api.yaml: Mapping for all implemented Flask endpoints.

Core Features Implemented

- Role-based authentication for Admin, Company, Student.
 - Single login form with backend role identification.
 - Company registration with Pending : Approved/Rejected admin workflow.
 - Admin dashboard for managing students, companies, drives, and applications.
 - Company can create/edit/close drives only after approval.
 - Company can update applicant status: Applied / Shortlisted / Selected / Rejected.
 - Student can view only valid approved drives and apply.
 - Blacklisting/deactivation via is_active (history preserved, no hard delete).
 - Profile view/edit available for all three roles.
 - Responsive Bootstrap UI, no JavaScript required for core actions.
-

8. Video Presentation

Drive Link:

 Recruit.mp4
